

1. Introduction

1.1. Purpose

Y.W. Acad System is a full-stack academic performance management system designed to store, retrieve, analyze, and mutate student grade records at scale. It provides real-time analytics, interactive dashboards, and event-driven streaming for monitoring academic performance across multiple departments and semesters.

1.2. Objective

- **Scalable data storage** — Leverage a sharded MongoDB cluster to distribute grade records across multiple shards, enabling horizontal read/write scalability.
- **Low-latency analytics** — Cache frequently queried department-level aggregations in Redis (60s TTL) with automatic cache invalidation on mutations.
- **Real-time event streaming** — Propagate grade mutations through Apache Kafka and broadcast them to connected clients via Server-Sent Events (SSE).
- **Interactive data exploration** — Provide a React-based single-page application with cursor-based pagination, multi-dimensional filtering, and Recharts-powered visualizations.
- **Operational tooling** — Include a data seeding script (350K records) and a live mutation simulator for development and stress testing.

1.3. Scope

The system consists of three main components:

Component	Technology	Role
Client	React 19 + Vite 8 + Recharts 3	Browser-based SPA dashboard
Server	Apollo Server 5 + Express 5 + MongoDB Driver 7	GraphQL API + SSE stream
Infrastructure	Docker Compose	MongoDB sharded cluster, Redis 7, Kafka + Zookeeper

The API exposes 8 queries and 2 mutations over a single GraphQL endpoint at <http://localhost:4000/graphql>, alongside an SSE stream at <http://localhost:4001/stream> for real-time event consumption.

1.4. Key Features

- **Sharded MongoDB Cluster** — Two shard replica sets fronted by a mongos router, designed for horizontal scaling of grade records.
- **Redis Caching Layer** — Distributed cache-aside pattern with 60-second TTL and explicit cache eviction on write operations.
- **Apache Kafka Event Bus** — Grade mutations are published to the grade-mutations Kafka topic and consumed to fan out events to all SSE clients.
- **Cursor-Based Pagination** — Stable, efficient pagination across sharded data using MongoDB ObjectId comparison (\$lt + descending sort).
- **Interactive Dashboard** — Five-tab SPA with KPI cards, bar/area/pie/line charts, sortable data tables, progressive search, inline grade editing, and a real-time event stream viewer.
- **Dark/Light Theme** — Full theme support persisted to localStorage via CSS custom properties.
- **Data Seeding & Simulation** — seed.js generates 350K grade records across 6 departments and 4 semesters; live-simulator.js runs random mutations every 15-20 seconds for end-to-end testing.

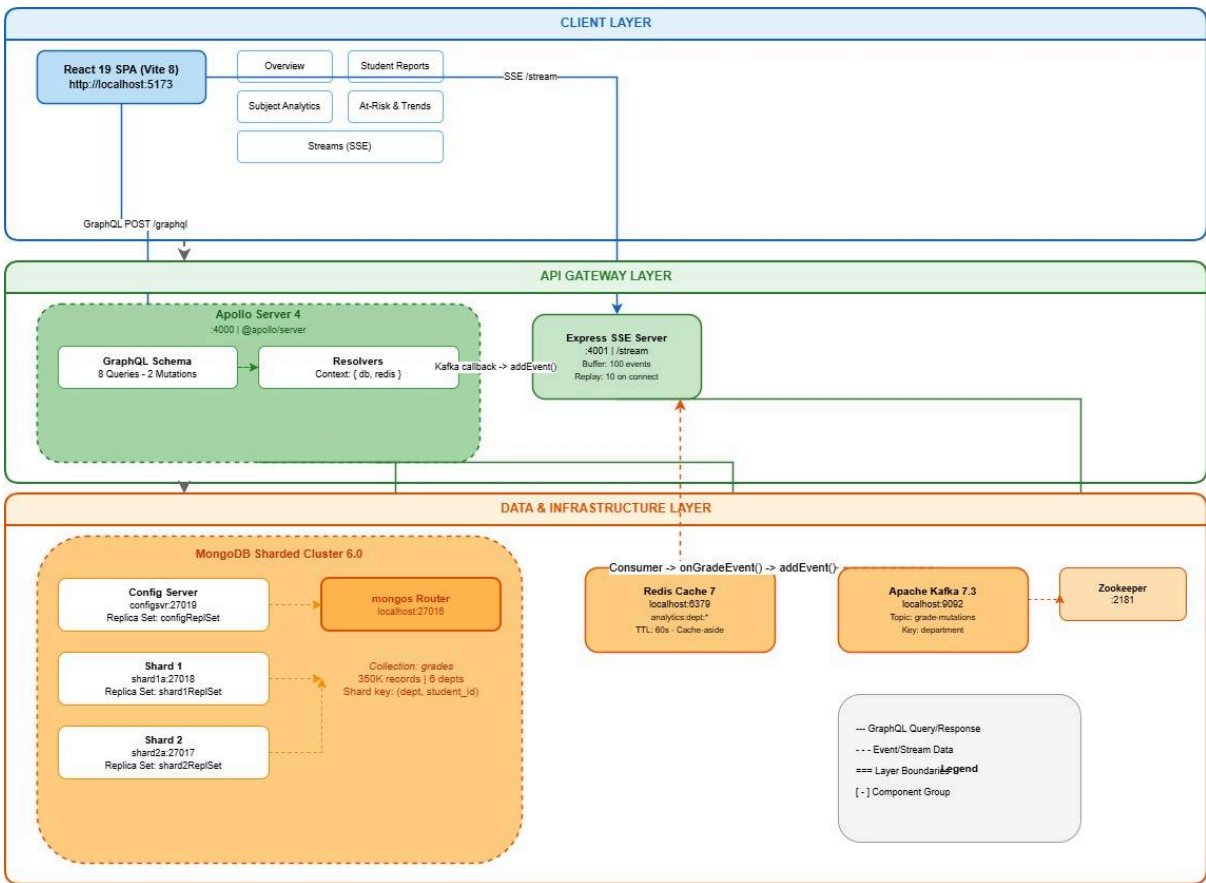
1.5. Apollo Server GraphQL Integration

The system uses Apollo Server 4 (@apollo/server) as its primary API layer, running as a standalone server on port 4000. The GraphQL schema is defined in server/src/graphql/typeDefs/gradeDefs.js and implemented by resolvers in server/src/graphql/resolvers/gradeResolvers.js. Key design decisions include:

- **Context-based dependency injection** — The MongoDB database handle and Redis client are passed through the Apollo context, making them available to every resolver without globals.
- **Cache-aware resolvers** — getDepartmentAnalytics implements a cache-aside pattern, checking Redis before querying MongoDB and setting a 60-second TTL on cache misses.
- **Write-through mutations** — Both updateStudentGrade and addGradeRecord follow a three-step pattern: (1) perform the MongoDB write, (2) evict the relevant Redis cache key, (3) publish the event to Kafka.
- **Performance instrumentation** — Every query and mutation tracks execution timing via performance.now(), returning totalTimeMs, dbQueryTimeMs, and cacheHit in the response for observability.

2. System Architecture

2.1. High-Level Architecture Diagram



2.2. Component Descriptions & Transactions

Apollo Server 4 serves as the single entry point for all data operations. It is started via `startStandaloneServer` in `server/src/index.js` on port 4000.

Request Flow:

1. The client sends an HTTP POST request to `/graphql` with a JSON body containing the query string.
2. Apollo parses the query against the schema defined in `gradeDefs.js` and delegates execution to the corresponding resolver in `gradeResolvers.js`.
3. Each resolver receives a context object injected at startup containing `{ db, redis }` — the MongoDB database handle and Redis client instance.
4. The resolver executes the appropriate data-fetching logic (MongoDB aggregation, Redis lookup, or both), transforms the result, and returns it.
5. Apollo serializes the response and sends it back to the client.

Resolver Delegation Pattern:

Query	Resolver Responsibility
<code>getGrades</code>	Builds a filtered MongoDB query with cursor-based pagination, fetches from the <code>grades</code> collection
<code>getDepartmentAnalytics</code>	Checks Redis cache first (<code>analytics:dept:*</code>); on miss, runs a MongoDB aggregation pipeline, caches result, returns
<code>getStudentGrades</code>	Direct MongoDB point query using compound shard key <code>{ department, student_id }</code>
<code>searchStudentById</code>	Prefix regex search across all departments with cursor-based pagination
<code>getSemestralAnalytics</code>	Aggregation pipeline grouped by semester
<code>getAtRiskCount</code>	Count query filtered by grade values (<code>\$in: [3.0, 5.0]</code>) and optional semester
<code>getGradeDistribution</code>	Aggregation pipeline grouped by grade
<code>getDepartmentsSemesterTrends</code>	Aggregation pipeline grouped by <code>{ department, semester }</code>
<code>updateStudentGrade</code>	MongoDB <code>findOneAndUpdate</code> , Redis cache eviction, Kafka event publish
<code>addGradeRecord</code>	

2.3.

2.4.

